Program to implement Leaky Bucket algorithm for traffic shaping

Aim: To Write C Program to implement Leaky bucket algorithm for Congestion control.

Description:

Traffic Shaping is a mechanism to control the amount and the rate of the traffic sent to the network. This approach of congestion management is called Traffic shaping. Traffic shaping helps to regulate rate of data transmission and reduces congestion..
There are 2 types of traffic shaping algorithms:

1. Leaky Bucket
2. Token Bucket

In networking, a technique called leaky bucket can smooth out bursty traffic. Bursty chunks are stored in the bucket and sent out at an average rate.

Algorithm:

1. Start
2. Set the bucket size or the buffer size.
3. Set the output rate.
4. Transmit the packets such that there is no overflow.
5. Repeat the process of transmission until all packets are transmitted. (Reject packets where its size is greater than the bucket size)
6. Stop

Program:

```
#include <stdio.h>

int main()
{
    int bucket_size, output_rate;
    int n, packet_size;
    int storage = 0;

    printf("Enter bucket capacity: ");
    scanf("%d", &bucket_size);

    printf("Enter output rate: ");
    scanf("%d", &output_rate);

    printf("Enter number of incoming packets: ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++)
    {
        printf("\nEnter size of packet %d: ", i);
        scanf("%d", &packet_size);

        printf("Packet size: %d\n", packet_size);

        if (packet_size > bucket_size - storage)
        {
            printf("Bucket overflow! Packet rejected.\n");
        }
        else
```

```
        {
            storage += packet_size;
            printf("Packet accepted.\n");
            printf("Bytes in bucket: %d\n", storage);
        }

        if (storage > 0)
        {
            int sent = (storage < output_rate)
                       ? storage : output_rate;

            storage -= sent;

            printf("Bytes sent: %d\n", sent);
            printf("Bytes remaining: %d\n", storage);
        }
    }

    while (storage > 0)
    {
        int sent = (storage < output_rate)
                   ? storage : output_rate;

        storage -= sent;

        printf("\nBytes sent: %d\n", sent);
        printf("Bytes remaining: %d\n", storage);
    }

    printf("\nAll packets processed.\n");

    return 0;
}
```

Output:

```
Enter bucket capacity: 10
Enter output rate: 4
Enter number of incoming packets: 3

Enter size of packet 1: 5
Packet size: 5
Packet accepted.
Bytes in bucket: 5
Bytes sent: 4
Bytes remaining: 1

Enter size of packet 2: 8
Packet size: 8
Packet accepted.
Bytes in bucket: 9
Bytes sent: 4
Bytes remaining: 5

Enter size of packet 3: 7
Packet size: 7
Bucket overflow! Packet rejected.
Bytes sent: 4
Bytes remaining: 1

Bytes sent: 1
Bytes remaining: 0

All packets processed.
```